



Programação WEB Avançada

Universidade de Vila Velha

Comprometida com a excelência no ensino, pesquisa e inovação.

✉ wanderson.santana@uvv.br

Capítulo 6

Framework CSS, Web Forms e Testes Automatizados

Resumo

Nesta unidade avançamos na construção da aplicação *delivery* incorporando três pilares fundamentais para aplicações web modernas: **formulários web seguros e usáveis**, o framework CSS **Bulma** para estilização rápida e responsiva, e **pytest** para testes automatizados. Iremos refatorar os templates para utilizar Bulma, implementar formulários com Flask-WTF + validação, proteger contra CSRF, e criar testes unitários e de integração para rotas e formulários. O objetivo é entregar uma interface profissional, formulários confiáveis e uma base de testes que garanta a qualidade do código à medida que a aplicação cresce.

“Um bom formulário não é apenas bonito: é seguro, validado e testável.”

Adaptado de Armin Ronacher e WTFForms

Introdução

Na unidade anterior consolidamos a arquitetura fundamental da aplicação Flask, adotando o padrão *Application Factory*, a organização modular via *Blueprints* e o uso de templates Jinja2 com herança. Também incorporamos ferramentas de apoio ao desenvolvimento, como a *Flask-DebugToolbar*, que nos auxiliam na inspeção do comportamento interno da aplicação.

Com essa base estrutural estabelecida, avançamos agora para uma nova camada de abstração: a **interface com o usuário**. Mais especificamente, passamos a tratar da *camada de apresentação*, responsável por transformar a lógica da aplicação em uma experiência visual e interativa consistente.

Esta unidade está organizada em torno de três pilares fundamentais:

- **Framework CSS (Bulma)**: responsável pela padronização visual e responsividade da interface;
- **Web Forms com Flask-WTF**: mecanismo estruturado para entrada, validação e segurança de dados do usuário;
- **Testes automatizados com pytest**: garantia de corretude e robustez das funcionalidades implementadas.

Do ponto de vista arquitetural, é importante observar que extensões no ecossistema Flask seguem um padrão consistente de inicialização, especialmente em aplicações que utilizam *Application Factory*.

Embora seja tecnicamente possível instanciar extensões diretamente passando o objeto da aplicação no construtor, a abordagem recomendada consiste em:

1. Declarar a extensão no escopo global, sem associação imediata com a aplicação;
2. Inicializá-la posteriormente por meio do método `init_app(app)` dentro da função fábrica.

Esse padrão promove:

- **Desacoplamento:** reduz dependências rígidas entre módulos;
- **Testabilidade:** facilita a criação de múltiplas instâncias da aplicação com diferentes configurações;
- **Evita importações circulares:** problema comum em aplicações mal estruturadas.

Uma vez inicializadas, essas extensões (como a `Flask-WTF`, que integra validação de formulários e proteção CSRF) passam a disponibilizar funcionalidades que são naturalmente integradas à camada de templates Jinja2.

Essa integração permite a construção de interfaces reutilizáveis, onde componentes visuais e comportamentais são definidos de forma modular e estendidos por herança — princípio que será amplamente explorado nesta unidade.

Adicionando Bulma — Um Framework CSS Leve e Moderno

Por que usar um framework CSS?

A construção manual de folhas de estilo CSS para aplicações completas tende a introduzir uma série de desafios:

- inconsistências visuais entre páginas;
- duplicação de código;
- dificuldade de manutenção;
- complexidade na implementação de layouts responsivos;
- necessidade de lidar com diferenças entre navegadores.

Frameworks CSS modernos surgem como uma solução sistemática para esses problemas, oferecendo:

- componentes pré-definidos;

- sistemas de grid e layout;
- utilitários de estilização;
- padrões visuais consistentes.

Neste contexto, optamos pelo **Bulma**, cujas características o tornam particularmente adequado para aplicações acadêmicas e projetos modulares em Flask:

- **Arquitetura baseada em CSS puro**: não depende de JavaScript, reduzindo complexidade e acoplamento;
- **Uso de Flexbox**: permite layouts responsivos de forma nativa e sem hacks;
- **Semântica clara**: classes com nomes descritivos facilitam a leitura e manutenção do código;
- **Customização via Sass**: possibilita adaptação da identidade visual;
- **Baixa curva de aprendizado**: favorece produtividade em ambientes educacionais;
- **Licença MIT**: livre para uso em qualquer contexto.

Essa escolha não é apenas estética, mas arquitetural: estamos adotando um **sistema de design reutilizável**, alinhado com o princípio de separação de responsabilidades.

Instalação e integração do Bulma

Como o Bulma é um framework exclusivamente CSS, sua integração é direta. Existem duas abordagens principais:

- **Via CDN**: recomendada para desenvolvimento e prototipação rápida;
- **Arquivos locais**: recomendada para produção, garantindo controle de versão e melhor desempenho.

No contexto atual, utilizaremos CDN para simplificação. No template base, inserimos:

Código: templates/main/base.html (trecho atualizado)

```
1 {% block head %}
2     <meta charset="UTF-8">
3     <meta name="viewport" content="width=device-width, initial-scale=1.0">
4     <title>{% block title %}Delivery UVV{% endblock %}</title>
5
6     {% block css %}
7     <link rel="stylesheet"
8         href="https://cdn.jsdelivr.net/npm/bulma@1.0.4/css/bulma.min.css">
9     {% endblock %}
10 {% endblock %}
```

NOTA

Em ambientes de produção, recomenda-se servir o arquivo CSS localmente para evitar dependência externa e melhorar o tempo de carregamento.

Refatorando os templates com Bulma

A introdução do Bulma exige a adaptação dos templates para utilizar suas classes utilitárias e componentes.

Essa refatoração não é meramente estética: ela representa a adoção de um **vocabulário visual padronizado**, no qual a estrutura HTML passa a expressar diretamente intenções de layout e estilo.

O template base torna-se:

Código: templates/main/base.html

```
1 <!DOCTYPE html>
2 <html lang="pt-br">
3 <head>
4     {% block head %}
5     <meta charset="UTF-8">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```

7 <title>{% block title %}Delivery UVV{% endblock %}</title>
8 <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bulma@1.0.4/css/bulma.
  min.css">
9 {% block css %}{% endblock %}
10 {% endblock %}
11 </head>
12 <body>
13   <header>
14     {% block navbar %}
15     {% include 'main/partials/navbar.html' %}
16     {% endblock %}
17   </header>
18
19   <section class="section">
20     <div class="container">
21       {% block content %}
22       {% endblock %}
23     </div>
24   </section>
25
26   <footer class="footer">
27     <div class="content has-text-centered">
28       {% block footer %}
29       {% include 'main/partials/footer.html' %}
30       {% endblock %}
31     </div>
32   </footer>
33
34   {% block scripts %}{% endblock %}
35 </body>
36 </html>

```

Observe que:

- section e container são componentes estruturais do Bulma;
- a hierarquia visual passa a ser definida por classes sem necessidade de CSS customizado;

- mantemos a extensibilidade via blocos Jinja2.

Refatoração da página inicial

Código: templates/main/index.html

```
1 {% extends 'main/base.html' %}
2
3 {% block title %}Home Page - Delivery UVV{% endblock %}
4
5 {% block content %}
6     {% if user %}
7         <h1 class="title is-3">
8             Ola, {{ user.username | capitalize }}!
9         </h1>
10        <p class="subtitle">
11            Bem-vindo ao sistema de delivery da UVV.
12        </p>
13    {% else %}
14        <h1 class="title is-3">Ola, visitante!</h1>
15        <p class="subtitle">
16            Voce precisa fazer login para continuar.
17        </p>
18    {% endif %}
19
20    {% if promocao %}
21        {% include 'main/partials/promocao.html' %}
22    {% endif %}
23 {% endblock %}
```

Aqui introduzimos:

- title e subtitle: padrões tipográficos do Bulma;
- melhoria semântica da hierarquia visual.

Refatoração do carrinho

Código: templates/main/carrinho.html

```

1 {% extends 'main/base.html' %}
2
3 {% block title %}{% titulo | default("Meu Carrinho") %}{% endblock %}
4
5 {% block content %}
6 {% import 'main/macros.html' as macros %}
7
8 <div class="container py-6">
9
10     <h1 class="title is-2 has-text-centered mb-6">
11         Meu Carrinho
12     </h1>
13
14     {% if carrinho.itens %}
15         <ul class="is-flex is-flex-direction-column" style="list-style: none;">
16             {% for item in carrinho.itens %}
17                 {% macros.item_carrinho(item) %}
18             {% endfor %}
19         </ul>
20
21         <div class="mt-5">
22             <strong>Total estimado:</strong>
23             <p class="title is-5 has-text-success has-text-weight-bold">
24                 R$ {% total | round(2) %}
25             </p>
26         </div>
27
28     {% else %}
29         <div class="notification is-danger has-text-centered py-6">
30             <p class="is-size-5">0 carrinho esta vazio.</p>
31             <p class="mt-2">Adicione itens para continuar.</p>
32         </div>
33     {% endif %}
34 </div>
35 {% endblock %}

```

Destacam-se aqui:

- uso de `notification`: componente pronto para mensagens;
- classes utilitárias para espaçamento e alinhamento;
- correção de inconsistência (`has-text-success`).

NOTA

A função `url_for` deve ser sempre utilizada para geração de URLs dinâmicas, garantindo independência da estrutura interna de rotas.

Web Forms com Flask-WTF

Formulários constituem o principal mecanismo de interação entre o usuário e a aplicação web. Embora o Flask ofereça suporte básico para manipulação de dados submetidos (via `request.form`), ele não fornece, de forma nativa, uma estrutura robusta para:

- definição declarativa de campos;
- validação sistemática de dados;
- proteção contra ataques de segurança;
- renderização integrada com templates.

Para suprir essas lacunas, utilizamos a extensão **Flask-WTF**, que atua como uma camada de integração entre o Flask e a biblioteca **WTForms**.

Além disso, utilizaremos o pacote `email-validator`, necessário para validação adequada de campos de e-mail.

```
pip install flask-wtf email-validator
```

NOTA

Não se esqueça de atualizar o arquivo `pyproject.toml`, garantindo reprodutibilidade do ambiente.

Configuração e proteção CSRF

A extensão Flask-WTF utiliza um mecanismo de segurança essencial chamado **CSRF** (*Cross-Site Request Forgery*), que protege a aplicação contra submissões maliciosas de formulários.

Esse mecanismo baseia-se na geração de **tokens criptográficos** associados à sessão do usuário. Para que esses tokens sejam seguros, é indispensável que a aplicação possua uma **chave secreta** configurada.

Na Unidade 05, já introduzimos um módulo de configuração utilizado, por exemplo, pela *Debug Toolbar*. Agora, essa mesma configuração passa a desempenhar um papel adicional: garantir a segurança dos formulários.

Código: `delivery/ext/config/__init__.py`

```
1 import os
2
3 def init_app(app):
4     app.config['SECRET_KEY'] = (
5         os.environ.get('SECRET_KEY')
6         or 'uma-chave-secreta-segura' # Nunca usar em producao
7     )
8
9     app.config['DEBUG'] = True
10
11     if app.debug:
12         app.config['DEBUG_TB_TEMPLATE_EDITOR_ENABLED'] = True
13         app.config['DEBUG_TB_INTERCEPT_REDIRECTS'] = False
```

NOTA

A `SECRET_KEY` é utilizada internamente pelo Flask e por extensões (como o Flask-WTF) para gerar assinaturas criptográficas.

Em produção, essa chave deve:

- ser única;
- possuir alta entropia (difícil de adivinhar);
- ser armazenada em variáveis de ambiente.

Aviso

O mecanismo de CSRF funciona inserindo automaticamente um token oculto em cada formulário. Esse token é validado no servidor a cada submissão, garantindo que a requisição foi realmente originada pela aplicação legítima.

Inicializando a extensão

Seguindo o padrão estabelecido na Unidade 05 (Application Factory), criamos um módulo dedicado à extensão:

Código: `delivery/ext/wtf/__init__.py`

```
1 from flask_wtf import CSRFProtect
2
3 csrf = CSRFProtect()
4
5 def init_app(app):
6     csrf.init_app(app)
```

E registramos na fábrica da aplicação:

Código: app.py

```
1 from flask import Flask
2 import logging
3
4 def create_app():
5     app = Flask(__name__)
6
7     if app.debug:
8         app.logger.setLevel(logging.DEBUG)
9
10    from delivery.ext.config import init_app as init_config
11    init_config(app)
12
13    from delivery.ext.wtf import init_app as init_wtf
14    init_wtf(app)
15
16    from delivery.ext.debugtoolbar import init_app as init_toolbar
17    init_toolbar(app)
18
19    from delivery.views import init_app as init_site
20    init_site(app)
21
22    return app
```

NOTA

Observe que a extensão é desacoplada da aplicação, sendo inicializada apenas dentro da factory — exatamente como recomendado para evitar dependências circulares.

Criando o primeiro formulário

Formulários em Flask-WTF são definidos como classes Python que herdam de `FlaskForm`. Cada campo é descrito como um atributo da classe.

Crie o módulo `delivery/forms/main.py`:

Código: delivery/forms/main.py

```
1 from flask_wtf import FlaskForm
2 from wtforms import StringField, TextAreaField, SubmitField, EmailField
3 from wtforms.validators import DataRequired, Email, Length
4
5 class ContatoForm(FlaskForm):
6     nome = StringField(
7         'Nome',
8         validators=[DataRequired(), Length(min=3, max=100)]
9     )
10
11     email = EmailField(
12         'E-mail',
13         validators=[DataRequired(), Email()]
14     )
15
16     mensagem = TextAreaField(
17         'Mensagem',
18         validators=[DataRequired(), Length(min=10)]
19     )
20
21     submit = SubmitField('Enviar Mensagem')
```

NOTA

Os **validators** encapsulam regras de validação. Caso alguma falhe, mensagens de erro são automaticamente associadas ao campo.

Usando o formulário na view**Código:** delivery/views/main.py

```
1 from flask import (
2     Blueprint, render_template, flash,
3     redirect, url_for, current_app
4 )
5
```

```

6 from delivery.forms.main import ContatoForm
7
8 bp_main = Blueprint("main", __name__)
9
10 @bp_main.route('/contato', methods=['GET', 'POST'])
11 def contato():
12     form = ContatoForm()
13
14     if form.validate_on_submit():
15         current_app.logger.info(
16             f"Mensagem recebida de {form.nome.data}"
17         )
18
19         flash('Mensagem enviada com sucesso!', 'success')
20         return redirect(url_for('main.index'))
21
22     return render_template('main/contato.html', form=form)

```

NOTA

O método `validate_on_submit()` combina duas verificações:

- se a requisição é do tipo POST;
- se todos os validadores do formulário foram satisfeitos.

Template do formulário com Bulma

Código: `templates/main/contato.html`

```

1 {% extends "main/base.html" %}
2
3 {% block content %}
4 <div class="container">
5     <div class="columns is-centered">
6         <div class="column is-8-tablet is-6-desktop">
7             <h1 class="title">Entre em Contato</h1>
8

```

```
9      {% with messages = get_flashed_messages(with_categories=true) %}
10      {% if messages %}
11          {% for category, message in messages %}
12              <div class="notification is-{{ category }}">
13                  {{ message }}
14              </div>
15          {% endfor %}
16      {% endif %}
17  {% endwith %}
18
19  <form method="POST" novalidate>
20      {{ form.hidden_tag() }}
21
22      <div class="field">
23          <label class="label">{{ form.nome.label }}</label>
24          <div class="control">
25              {{ form.nome(class="input") }}
26          </div>
27          {% for error in form.nome.errors %}
28              <p class="help is-danger">{{ error }}</p>
29          {% endfor %}
30      </div>
31
32      <!-- Repetir para email e mensagem -->
33
34      <div class="field">
35          <div class="control">
36              {{ form.submit(class="button is-primary is-fullwidth") }}
37          </div>
38      </div>
39  </form>
40  </div>
41  </div>
42 </div>
43 {% endblock %}
```


⚠ Aviso

A chamada `{{ form.hidden_tag() }}` é obrigatória para incluir o token CSRF. Sem ela, a aplicação ficará vulnerável a ataques.

Aspectos conceituais importantes

- **Separação de responsabilidades:** definição do formulário (Python), renderização (Jinja2) e processamento (view);
- **Validação no servidor:** garante integridade dos dados independentemente do cliente;
- **Proteção CSRF:** baseada em tokens criptográficos dependentes da `SECRET_KEY`;
- **Reutilização:** formulários podem ser facilmente reutilizados em múltiplas rotas.

Testes Automatizados com pytest

Testes automatizados são fundamentais para garantir a confiabilidade da aplicação. À medida que novas funcionalidades são adicionadas, os testes asseguram que comportamentos previamente implementados não sejam inadvertidamente quebrados — princípio conhecido como *regression safety*.

Além disso, testes bem estruturados permitem:

- evolução segura do código;
- documentação executável do sistema;
- detecção precoce de erros;
- maior confiança em refatorações.

Gerenciamento de ambientes com dotenv

Antes de configurar o `pytest`, precisamos estruturar corretamente os ambientes da aplicação. Para isso, utilizaremos a biblioteca `python-dotenv`, que permite carregar variáveis de ambiente a partir de arquivos.

Separação de responsabilidades: `.flaskenv` vs `.env`

É importante distinguir dois tipos de arquivos:

- `.flaskenv`: utilizado pelo comando `flask` para variáveis de inicialização da aplicação (ex: `FLASK_APP`, `FLASK_DEBUG`);
- `.env` (ou variantes): utilizado para configurações da aplicação, incluindo dados sensíveis (ex: `SECRET_KEY`, credenciais de e-mail, etc.).

NOTA

O arquivo `.flaskenv` pode ser versionado no repositório, pois não contém informações sensíveis. Já arquivos `.env` **não devem ser versionados**, pois podem conter segredos da aplicação.

Estratégia adotada no projeto

Para suportar múltiplos ambientes, adotaremos a seguinte convenção:

- `.env.dev`
- `.env.test`
- `.env.prod`

Esses arquivos armazenam configurações específicas de cada ambiente.

Código: .env.dev

```
1 FLASK_APP=app.py
2 FLASK_ENV=development
3 FLASK_DEBUG=1
4
5 SECRET_KEY=dev-secret-key
```

Código: .env.test

```
1 FLASK_APP=app.py
2 FLASK_ENV=testing
3 FLASK_DEBUG=0
4
5 SECRET_KEY=test-secret-key
6 TESTING=1
7 WTF_CSRF_ENABLED=0
```

Código: .env.prod

```
1 FLASK_APP=app.py
2 FLASK_ENV=production
3 FLASK_DEBUG=0
4
5 SECRET_KEY=chave-super-segura
6
7 MAIL_SERVER=smtp.gmail.com
8 MAIL_PORT=587
9 MAIL_USE_TLS=True
10 MAIL_USERNAME=uvvweb@gmail.com
11 MAIL_PASSWORD=<senha-de-aplicativo>
12 MAIL_DEFAULT_SENDER=('UVV Web', 'uvvweb@gmail.com')
```

** Aviso**

Nunca versionar arquivos .env contendo credenciais reais. Utilize variáveis de ambiente no servidor de produção.

Carregamento automático das variáveis

O comando `flask run` já realiza automaticamente a leitura dos arquivos `.flaskenv` e `.env`. No entanto, em cenários como:

- execução via scripts personalizados;
- uso de `pytest`;
- execução em produção (sem o comando `flask`);

é necessário carregar explicitamente as variáveis de ambiente.

Para isso, integramos o carregamento ao módulo de configuração:

Código: `delivery/ext/config/__init__.py`

```
1 import os
2 from dotenv import load_dotenv
3
4 def init_app(app):
5     # Carrega variaveis do ambiente (ja selecionado via tasks.py)
6     load_dotenv(override=True)
7
8     app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY')
9     app.config['DEBUG'] = os.environ.get('FLASK_DEBUG') == '1'
10
11     app.config['MAIL_SERVER'] = os.environ.get('MAIL_SERVER')
12     app.config['MAIL_PORT'] = os.environ.get('MAIL_PORT')
13     app.config['MAIL_USE_TLS'] = os.environ.get('MAIL_USE_TLS') == 'True'
14     app.config['MAIL_USERNAME'] = os.environ.get('MAIL_USERNAME')
15     app.config['MAIL_PASSWORD'] = os.environ.get('MAIL_PASSWORD')
16     app.config['MAIL_DEFAULT_SENDER'] = os.environ.get('MAIL_DEFAULT_SENDER')
17
18     if app.debug:
19         app.config['DEBUG_TB_TEMPLATE_EDITOR_ENABLED'] = True
20         app.config['DEBUG_TB_INTERCEPT_REDIRECTS'] = False
```

NOTA

O arquivo carregado depende do ambiente previamente definido (por exemplo, via `tasks.py`). Isso permite total controle sobre qual configuração está ativa.

Considerações importantes

- Variáveis como `FLASK_APP` e `FLASK_DEBUG` são utilizadas no processo de inicialização do Flask e devem estar disponíveis antes da criação da aplicação;
- Variáveis sensíveis (ex: `SECRET_KEY`, credenciais) devem ser carregadas via `.env`;
- O uso de múltiplos arquivos `.env.*` permite isolar completamente os ambientes;
- O carregamento explícito com `python-dotenv` garante compatibilidade fora do comando `flask`.

NOTA

Essa abordagem segue práticas modernas de desenvolvimento, alinhadas com o princípio de *configuração externa* (*externalized configuration*), amplamente utilizado em aplicações escaláveis.

Automatizando com `tasks.py`

Com a definição de múltiplos arquivos `.env`, torna-se necessário garantir que o ambiente correto seja carregado antes da execução de qualquer comando.

Para isso, utilizamos o arquivo `tasks.py`, baseado na biblioteca `invoke`, como um ponto central de automação.

A estratégia adotada consiste em:

- carregar explicitamente o arquivo `.env.<ambiente>` desejado;
- executar o comando correspondente (execução, testes, etc.);

- garantir isolamento entre os ambientes.

A seguir, apresentamos a versão atualizada do `tasks.py` com suporte a múltiplos ambientes:

Código: tasks.py (trecho relevante)

```
1 from invoke import task
2 from datetime import datetime
3 from dotenv import load_dotenv
4 import os
5 import zipfile
6
7
8 def load_env(env: str):
9     """
10     Carrega o arquivo .env correspondente ao ambiente.
11     Ex: dev, test, prod
12     """
13     env_file = f".env.{env}"
14
15     if os.path.exists(env_file):
16         load_dotenv(env_file, override=True)
17         print(f"[ENV] Carregado: {env_file}")
18     else:
19         raise FileNotFoundError(f"{env_file} nao encontrado")
20
21
22 @task
23 def run(c):
24     """
25     Executa a aplicacao em ambiente de desenvolvimento.
26     """
27     load_env("dev")
28     c.run("flask run", pty=True)
29
30
31 @task
32 def test(c):
```

```

33     """
34     Executa os testes automatizados em ambiente isolado.
35     """
36     load_env("test")
37     c.run("pytest tests -v", pty=True)
38
39
40 @task
41 def prod(c):
42     """
43     Executa a aplicacao em modo producao.
44     """
45     load_env("prod")
46     c.run("flask run", pty=True)

```

⚠ Aviso

O uso de `override=True` garante que variáveis previamente definidas sejam substituídas pelo ambiente selecionado, evitando inconsistências.

NOTA

Essa abordagem permite reproduzir localmente diferentes cenários (desenvolvimento, testes e produção) com alto grau de fidelidade, sem necessidade de alterar o código da aplicação.

Configuração do pytest

Adicione ao `pyproject.toml`:

```

1 [tool.pytest.ini_options]
2 testpaths = ["tests"]
3 addopts = "-v"

```

Agora podemos criar nossos testes.

Exemplo de testes básicos

Nesta seção apresentamos um conjunto inicial de testes automatizados que cobrem diferentes aspectos da aplicação:

- criação da aplicação (infraestrutura);
- carregamento de configuração;
- comportamento de rotas;
- validação de respostas HTTP;
- fluxo de envio de formulários.

O objetivo não é apenas verificar funcionamento, mas também introduzir boas práticas de teste desde o início do desenvolvimento.

Código: tests/test_main.py

```
1 import pytest
2 from delivery import create_app
3
4
5 # =====
6 # FIXTURES (infraestrutura de teste)
7 # =====
8
9 @pytest.fixture
10 def app():
11     """
12     Cria uma instancia da aplicacao configurada para testes.
13     """
14     app = create_app()
15     app.config.update({
16         "TESTING": True,
17         "WTF_CSRF_ENABLED": False, # desabilita CSRF nos testes
18     })
19     return app
```



```
20
21
22 @pytest.fixture
23 def client(app):
24     """
25     Cliente HTTP de teste (simula requisicoes sem servidor real).
26     """
27     return app.test_client()
28
29
30 # =====
31 # TESTES DE INFRAESTRUTURA
32 # =====
33
34 def test_app_is_created(app):
35     """
36     Verifica se a aplicacao foi criada corretamente.
37     """
38     assert app is not None
39
40
41 def test_config_is_loaded(app):
42     """
43     Garante que as configuracoes de teste foram aplicadas.
44     """
45     assert app.config["TESTING"] is True
46
47
48 def test_app_is_running_in_test_mode(app):
49     """
50     Confirma explicitamente o modo de execucao da aplicacao.
51     """
52     assert app.config["TESTING"] is True
53
54
55 # =====
56 # TESTES DE ROTAS
57 # =====
58
```

```
59 def test_non_existent_route_returns_404(client):
60     """
61     Rotas inexistentes devem retornar HTTP 404.
62     """
63     response = client.get("/url_que_nao_existe")
64     assert response.status_code == 404
65
66
67 # -----
68 # TESTES PARAMETRIZADOS DE ROTAS PUBLICAS
69 # -----
70
71 PUBLIC_ROUTES = [
72     ("/", True),
73     ("/sobre", True),
74     ("/pesquisas", True),
75 ]
76
77
78 @pytest.mark.parametrize("route, check_doctype", PUBLIC_ROUTES)
79 def test_public_routes_return_200_in_html(client, route, check_doctype):
80     """
81     Verifica que rotas publicas:
82     - retornam status 200
83     - entregam HTML valido (verificacao minima)
84
85     O uso de parametrizacao permite escalar facilmente os testes.
86     """
87     response = client.get(route)
88
89     assert response.status_code == 200, \
90         f"Rota {route} retornou {response.status_code}"
91
92     assert (
93         b"<html" in response.data or
94         b"<!DOCTYPE html>" in response.data
95     ), f"Rota {route} nao parece retornar HTML"
96
97     if check_doctype:
```

```

98     assert b"<!DOCTYPE html>" in response.data, \
99         f"Rota {route} deveria conter DOCTYPE html"
100
101
102 # =====
103 # TESTES DE CONTEUDO
104 # =====
105
106 def test_index_page(client):
107     """
108     Verifica conteudo minimo da pagina inicial.
109     """
110     response = client.get('/')
111     assert response.status_code == 200
112     assert b"Delivery UVV" in response.data
113
114
115 # =====
116 # TESTES DE FORMULARIO
117 # =====
118
119 def test_contato_form(client):
120     """
121     Testa o fluxo completo de envio de formulario valido:
122     - POST com dados corretos
123     - espera redirecionamento (302)
124     """
125     response = client.post('/contato', data={
126         "nome": "Teste Aluno",
127         "email": "teste@uvv.br",
128         "mensagem": "Mensagem de teste para validacao"
129     })
130
131     assert response.status_code == 302 # redirect apos sucesso

```

Discussão dos testes

Cada grupo de testes possui um propósito específico dentro da estratégia de validação da aplicação.

Fixtures

As **fixtures** são responsáveis por preparar o ambiente de teste:

- `app`: cria a aplicação em modo de teste;
- `client`: fornece um cliente HTTP para simular requisições.

Essa abordagem evita repetição de código e garante isolamento entre os testes.

Testes de infraestrutura

- Garantem que a aplicação é criada corretamente;
- Validam que o modo de teste está ativo;
- Evitam erros silenciosos de configuração.

Testes de rotas

- Verificam códigos HTTP corretos (200, 404, etc.);
- Garantem que endpoints respondem conforme esperado;
- São fundamentais para APIs e aplicações web.

Parametrização de testes

O uso de `@pytest.mark.parametrize` permite:

- reduzir duplicação de código;

- testar múltiplas rotas com uma única função;
- facilitar expansão futura.

Validação de HTML

A verificação de trechos como `<!DOCTYPE html>` é uma forma simples de garantir que:

- o template foi renderizado corretamente;
- não houve retorno inesperado (JSON, erro, etc.).

Testes de conteúdo

Verificar strings específicas (como “Delivery UVV”) assegura que:

- o conteúdo esperado está presente;
- o template correto foi utilizado.

Testes de formulário

O teste de formulário valida um fluxo completo:

1. envio de dados via POST;
2. validação dos campos;
3. execução da lógica da view;
4. redirecionamento após sucesso.

NOTA

Durante os testes, o CSRF é desabilitado para simplificar a simulação de re-

quisições. Em produção, essa proteção deve permanecer sempre ativa.

Visão geral

Esse conjunto de testes ilustra uma estratégia progressiva:

- começar pela infraestrutura;
- validar rotas básicas;
- verificar conteúdo;
- evoluir para fluxos completos.

Essa abordagem permite construir uma base sólida de testes que pode ser expandida ao longo do desenvolvimento da aplicação.

NOTA

O método `app.test_client()` permite simular requisições HTTP sem a necessidade de iniciar um servidor real, tornando os testes rápidos, determinísticos e isolados.

Testando templates e formulários

Os testes podem ser estendidos para validar:

- renderização correta de templates;
- presença de mensagens de erro;
- comportamento de validação de formulários;
- fluxos completos (ex: envio → validação → redirecionamento).

Por exemplo, podemos testar falhas de validação:

```
1 def test_contato_form_invalido(client):
2     response = client.post('/contato', data={
3         "nome": "",
4         "email": "email-invalido",
5         "mensagem": "curta"
6     })
7     assert response.status_code == 200 # nao redireciona
8     assert b"error" in response.data.lower()
```

Boas práticas combinadas

- Utilize arquivos `.env` para separar configurações por ambiente;
- Nunca exponha a `SECRET_KEY` em produção;
- Sempre use `url_for` para geração de URLs;
- Mantenha formulários em um pacote dedicado (`forms/`);
- Valide dados no servidor — nunca confie apenas no cliente;
- Escreva testes para rotas críticas e fluxos principais;
- Utilize fixtures do `pytest` para reaproveitamento de código;
- Mantenha consistência visual com o Bulma;
- Preserve a simplicidade e extensibilidade do template base.

Encerramento da Unidade

Nesta unidade, avançamos significativamente na construção da aplicação ao incorporar:

- uma camada de apresentação moderna com Bulma;
- formulários robustos e seguros com Flask-WTF;

- uma estratégia de testes automatizados com pytest;
- e um gerenciamento estruturado de ambientes.

Durante a execução dos testes, o envio do formulário de contato não resultava no redirecionamento esperado (HTTP 302), mas sim na re-renderização da página (HTTP 200).

Após investigação, identificou-se que o método `validate_on_submit()` retornava `False`, impedindo a execução do fluxo principal.

O problema estava em um erro de nomenclatura no formulário:

- No formulário: `mensagem`
- No teste: `mensagem`

Esse desalinhamento fez com que o campo obrigatório não fosse reconhecido, resultando na falha de validação.

Aviso

Erros de nomenclatura em formulários não geram exceções explícitas, mas alteram silenciosamente o fluxo da aplicação.

Na próxima unidade, integraremos persistência de dados com SQLAlchemy e mecanismos de autenticação, consolidando uma aplicação web completa e escalável.

Lista de Exercícios

6.1 Por que é recomendável usar um framework CSS como Bulma?

- (A) Para escrever mais CSS manualmente
- (B) Para acelerar o desenvolvimento, garantir responsividade e consistência visual
- (C) Porque o Flask não suporta CSS
- (D) Apenas para aplicações mobile

6.2 Qual é a principal vantagem do Flask-WTF?

- (A) Substitui o Jinja2
- (B) Remove a necessidade de templates
- (C) É obrigatório para usar Bulma
- (D) Integra WTForms ao Flask com proteção CSRF automática

6.3 O que o método `form.validate_on_submit()` faz?

- (A) Apenas verifica se o formulário existe
- (B) Renderiza o template automaticamente
- (C) Verifica se o formulário foi submetido via POST e se todos os validadores passaram
- (D) Salva os dados no banco

6.4 Por que incluimos `{{ form.csrf_token }}` no template?

- (A) Para proteger contra ataques Cross-Site Request Forgery (CSRF)
- (B) Para estilizar o formulário
- (C) Para enviar o formulário via AJAX
- (D) Para validar o e-mail automaticamente

6.5 Qual fixture do pytest é usada para criar um cliente de teste?

- (A) `app`
- (B) `client`
- (C) `browser`
- (D) `request`

6.6 Qual afirmação sobre testes com Flask está correta?

- (A) Testes devem sempre subir o servidor real
- (B) Não é possível testar formulários
- (C) `pytest` não funciona com Flask
- (D) Devemos desabilitar CSRF nos testes com `WTF_CSRF_ENABLED = False`

6.7 O que `url_for('main.index')` faz?

- (A) Gera a URL correta para a rota `index` do blueprint `main`
- (B) Redireciona o usuário
- (C) Renderiza o template
- (D) Cria uma nova rota